Multi-Agent Systems & Agentic AI — Practical Examination

Part I — GRPO finetuning of `gemma-3-1b-it` on GSM8K

Part II — adaptive planning in `cmbagent_lg`

Team **chmawa** (sm3035, bc654, zw499) · written individually by Zixuan Wang (zw499)

Jointly owned (shared team work): the v6e-1 TPU, the six shared GRPO runs R0–R5, and the code repository. All prose, figure selection and interpretation are my own.

Code: `github.com/wdk0082/a8-grpo` *(to be ported to GitLab for submission)* | W&B: `a8-grpo`

References: GRPO, DAPO, LoRA, GSM8K

Contents

Part I — GRPO Finetuning of `gemma-3-1b-it` on GSM8K

I.1 Reproducing the baseline

(i) The run. We reproduce the baseline as run **R0** (`8c2785ut`): upstream commit `borisbolliet/tpu-2026@77c5a67` with the `config.py` defaults (β=0.08, G=2), full **3364** GRPO steps (one epoch), **∼5 h 05 m** wall-clock on `v6e-1`.

(ii) Accuracy. Held-out GSM8K *test*, HF split, fixed seed 42, greedy/exact-match, reported as *(*n=64 */*n=1319*)* where n=64 is the `config.py` default and n=1319 is the full test: base **45.3 / 47.4%**; the LoRA-finetuned checkpoint scores **42.2 / 46.7%** at best-val (step 700) and **0.0 / 6.2%** at the final step 3364 (per-checkpoint CIs in Table 1).

(iii) Training curves. Fig. 1 (R0, black): mean reward $\bar{r}$ rises, peaks ≈step 1000, then declines; $\mathrm{KL}(\pi_\theta \| \pi_{\mathrm{ref}})$ climbs from 0 (an early spike to ∼4, settling ∼1.6).

Baseline patches (fixes for what did not run as-shipped; full diff in `tpu-2026_our_changes.patch`). (1) the W&B entity defaulted to a third party — redirected to our `a8-grpo` project; (2) `evaluate.py` built a fresh B=0 LoRA and never restored a checkpoint, so it could only score *base* — we added Orbax `-step`/`-ckpt-dir` LoRA restore; (3) TFDS's gsm8k builder crashes under protobuf 7.x once JAX imports, so we evaluate the identical GSM8K test via an HF `datasets` fallback (seed 42); (4) raised checkpoint retention (`SAVE_INTERVAL_STEPS` 500→100, `MAX_TO_KEEP` 4→40) so the best-val checkpoint survives selection (persistence, not training).

I.2 Understanding the baseline

(a) Policies & trainable parameters. π_θ is the LoRA-adapted policy — frozen `gemma-3-1b-it` weights plus low-rank adapters $\Delta W = BA$ — and is **the only policy carrying trainable parameters** (A, B; the base weights are frozen). π_{old} is the policy that generated the current rollouts; with `NUM_ITERATIONS` μ=1 there is one gradient step per rollout batch, so $\pi_{\mathrm{old}}=\pi_\theta$ at the update. π_{ref} is **the frozen base model** `gemma-3-1b-it` — equivalently π_θ with the adapters disabled — used as the fixed reference for the KL penalty; since LoRA initialises B=0, $\pi_\theta=\pi_{\mathrm{ref}}$ exactly at step 0.

(b) Group size & estimator. K=`NUM_GENERATIONS`= 2. Tunix uses `advantage_estimator='grpo'`: the **standard** group-mean, std-normalised advantage $\hat{A}_i=(r_i-\bar{r})/\sigma_r$ with $\bar{r}, \sigma_r$ over all K samples *including* i — *not* leave-one-out.

(c) Shaping vs. correctness rewards. Shaping: `match_format_exactly`, `match_format_approximately` (the `<reasoning>`/`<answer>` template). Correctness: `check_answer` (gold match, only if the full template parses) and its looser fallback `check_numbers` (any number after `<answer>`). With correctness *alone* σ_r vanishes on most groups — not because the policy is rarely right (base scores 47%, Table 1) but because the reward is *coarse*: at K=2 the two siblings usually *tie*, splitting only when exactly one is parseable-and-correct. A tie gives $r_i-\bar{r}$=0 over σ_r+10^{-6} (Tunix, `algo_core.py`), so $\hat{A}_i$=0 *exactly*: no signal, not an unstable 0/0; early learning is slow. The dense shaping terms keep $\sigma_r>0$ while giving gradient signal toward parseable outputs (the precondition for correctness to vary), so the exact-match reward can actually take effect in later training steps.

(d) Clipped surrogate & ϵ. The PPO-style clip lives in Tunix's `GRPOLearner` (module `tunix.rl`); the clip range is ϵ=`EPSILON`= 0.2 in `scripts/config.py`. Two deviations from the lecture form. *(i) Token- vs sequence-level:* the lectures clip a **per-response** ratio $\rho_i=\pi_\theta(a_i \mid q)/\pi_{\mathrm{old}}(a_i \mid q)$ (whole-sequence likelihood, one clip per response); Tunix clips a **per-token** ratio $\rho_{i,t}=\pi_\theta(a_{i,t} \mid q, a_{i,<t})/\pi_{\mathrm{old}}(a_{i,t} \mid q, a_{i,<t})$ token-by-token (a token-level objective, as in DAPO). *(ii) The clip is inert in this baseline:* with μ=`NUM_ITERATIONS`= 1 there is one gradient step per rollout batch, so $\pi_{\mathrm{old}}=\pi_\theta$, every $\rho \equiv 1$, and nothing is ever clipped — the surrogate collapses to the plain (unclipped) policy gradient and ϵ is irrelevant. This is not a *contradiction* (the lecture form also reduces to this at μ=1), but a non-trivial hyperparameter-induced consequence: the baseline's "clipped" objective performs no clipping at all; the clip (and ϵ) would bind only once $\mu > 1$.

I.3 Improving on the baseline

Modification & motivation.

Table 1: Held-out accuracy on the *full* GSM8K test (n=1319, HF split, seed 42), greedy/exact-match, ±95% bootstrap CI; each run trained for the full **3364** GRPO steps (one epoch), scored at its *best-val* (the saved checkpoint nearest the peak held-out validation *reward*) and *final* (step 3364) checkpoints. Last column: *paired* bootstrap Δ vs. base over the shared prompts (`paired_ci.py`). $*$ = paired 95% CI excludes 0; † = collapse. The β-axis extension R1(G=2, β=.3) fell *below* base (best 37.2 / final 0.0) — a clean negative result. All five runs are logged to the W&B project `a8-grpo`.

Model	(G, β), step	Acc. (%)	95% CI	Δ base (paired)
Base `gemma-3-1b-it`	—	47.4	44.7–50.0	—
R0 (baseline)	(2, .08) best 700	46.7	44.0–49.4	−0.7 n.s.
R0	(2, .08) final 3364	6.2	4.9–7.5	−41.2†
R4	(2, 0) best 1300	52.8	50.1–55.5	+5.4*
R4	(2, 0) final 3364	48.6	45.9–51.3	+1.2 n.s.
R3b	(8, 0) best 1000	55.7	53.0–58.2	+8.3*
R3b	(8, 0) **final 3364**	**57.2**	54.6–59.9	+9.9*
R5	(8, .08) best 1300	53.4	50.7–56.2	+6.1*
R5	(8, .08) final 3364	55.6	52.8–58.2	+8.2*

(i) Why R0 fails. R0 over-optimises at the smallest group size K=2: the lecture group-mean estimator $\hat{g} = \frac{1}{K}\sum_i h_i \hat{A}_i$ has gradient variance

$$\mathrm{Var}(\hat{g}) = \tfrac{1}{K^2}\textstyle\sum_i (h_i - \bar{h})(h_i - \bar{h})^\top = O(1/K),$$

maximal at K=2, so the noisy updates let the policy chase the dense *shaping* reward (reward-hacking the format) while true correctness decays.

(ii) Predictions from this theory. (1) raising the group size G shrinks $\mathrm{Var}(\hat{g}) \propto 1/K$, predicting smoother updates and milder over-optimisation; (2) the reference-KL penalty pulls the update toward π_{ref} — the penalised optimum is $\pi^\star(a) \propto \pi_{\mathrm{ref}}(a)\, e^{\hat{A}(a)/\beta} \to \pi_{\mathrm{ref}}$ as β grows — so a *tighter* leash (larger β) should curb the drift. We test both with a controlled $\beta \times G$ **2×2** — R4(G=2, β=0), R0(G=2, β=.08; baseline), R3b(G=8, β=0), R5(G=8, β=.08) — plus a β-axis extension R1(G=2, β=.3), for **five full runs** sharing data (HF GSM8K, seed 42), eval, horizon (3364 steps) and rollout RNG (`PRNGKey(0)`); only the named knob changes.

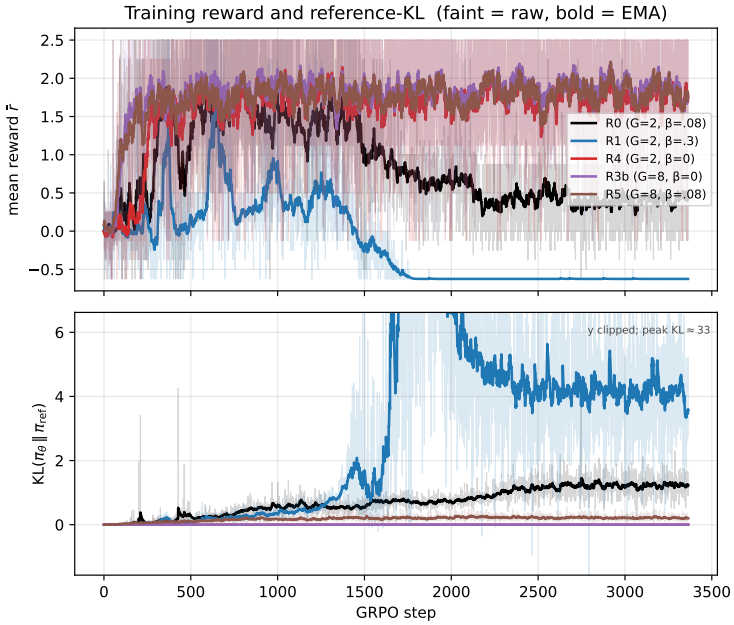


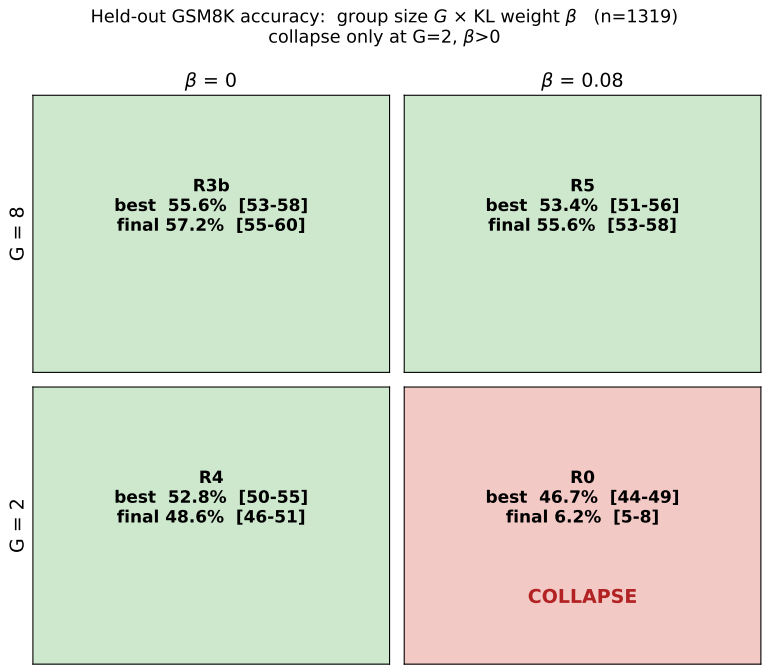
Figure 1: Mean training reward $\bar{r}$ (top) and $\mathrm{KL}(\pi_\theta \| \pi_{\mathrm{ref}})$ (bottom) vs. GRPO step, for all five runs on shared axes. R0 over-optimises (reward peaks then declines) and R1 collapses (KL blow-up ∼step 1700); the G=8 runs (R3b, R5) and R4 stay stable.

(a) Accuracy (Table 1): base, the baseline R0, and the improved checkpoints on the full test (n=1319,

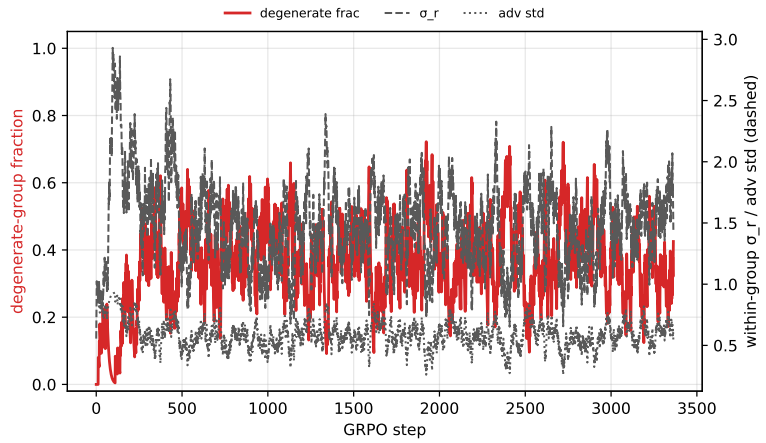
seed 42), with ±95% bootstrap and paired-Δ CIs over the shared prompts.

(b) Training curves (Fig. 1): mean reward and KL for the five runs on shared axes.

(c) Diagnostic (Fig. 2b): group degeneracy at G=8 — about a third of groups are all-K-equal (zero advantage) yet within-group σ_r holds ≈ 1.5, not $\to 0$.



(a) $\beta \times G$ 2×2 (best/final acc).



(b) Group degeneracy at G=8.

Figure 2: *Left:* accuracy by (G, β) — collapse occurs *only* at G=2, β>0 (R0). *Right:* the diagnostic — at G=8 about a third of groups are degenerate (all-K rewards equal) yet σ_r holds ≈ 1.5 (steady, not $\to 0$).

(d) Findings vs. prediction. *Prediction (1) holds.* G=8 is the winning intervention: **R3b (G=8, β=0) is the best model at 57.2%** (+9.9 pp over base, paired CI excludes 0); both G=8 runs beat base and each beats its matched G=2 run, and the 2×2 (Fig. 2a) shows over-optimisation is a G=2 phenomenon — G=2 peaks then degrades (R4 52.8→48.6; R0 collapses) while G=8 improves monotonically to the final step — exactly as the lower Var($\hat{g}$) predicts. *Prediction (2) is contradicted.* The tighter KL leash did not help: at G=2, β=0 (R4) beats β=0.08 (R0, which collapses) and pushing to β=0.3 (R1) collapsed *faster* (final 0%); the collapse is confined to the G=2, β>0 corner, and at G=8 the KL weight is statistically irrelevant (R3b vs. R5: +1.7 pp, n.s.). The cure was variance reduction (group size), not regularisation — echoing DAPO's removal of the KL term for long-output RL. The cost of G=8 is group degeneracy (Fig. 2b, the dead-group / effective-sample-size issue), but the net effect is still +8–10 pp.

I.4 GRPO theory — written questions (40 marks)

I.4.1 Group-mean normalisation

Let

$$\bar{h} = \frac{1}{K}\sum_{i=1}^{K} h_i.$$

(i) Since $\mathbb{E}[r_i] = \mathbb{E}[\bar{r}] = \mu$,

$$\mathbb{E}[\hat{A}_i] = \frac{\mathbb{E}[r_i - \bar{r}]}{\sigma} = 0.$$

Because $X_i = h_i \hat{A}_i$ and h_i is fixed, linearity of expectation gives

$$\mathbb{E}[X_i] = h_i\,\mathbb{E}[\hat{A}_i] = h_i \cdot 0 = 0, \qquad \mathbb{E}[X_i] = 0.$$

Summing over the group and again using linearity,

$$\mathbb{E}[\hat{g}] = \mathbb{E}\left[\frac{1}{K}\sum_{i=1}^{K} X_i\right] = \frac{1}{K}\sum_{i=1}^{K}\mathbb{E}[X_i] = \frac{1}{K}\sum_{i=1}^{K} 0 = 0, \qquad \mathbb{E}[\hat{g}] = 0.$$

Every expectation in this question is taken conditional on the sampled responses $a_1, \ldots, a_K$, which are held fixed; that is, $\mathbb{E}[\,\cdot\,]$ abbreviates $\mathbb{E}[\,\cdot \mid a_1, \ldots, a_K]$, so each score h_i is constant and only the artificial reward noise is random. In the usual policy-gradient problem the responses are sampled from the policy and their rewards depend on the sampled responses, so the policy gradient need not vanish.

(ii) The common random term is the sample mean $\bar{r}$, appearing in every

$$X_i = \frac{h_i}{\sigma}(r_i - \bar{r}).$$

For $i \neq j$,

$$\begin{aligned}\operatorname{Cov}(r_i - \bar{r}, r_j - \bar{r}) &= \operatorname{Cov}(r_i, r_j) - \operatorname{Cov}(r_i, \bar{r}) - \operatorname{Cov}(\bar{r}, r_j) + \operatorname{Var}(\bar{r}) \\ &= 0 - \frac{\sigma^2}{K} - \frac{\sigma^2}{K} + \frac{\sigma^2}{K} \\ &= -\frac{\sigma^2}{K}.\end{aligned}$$

Hence

$$\operatorname{Cov}(X_i, X_j) = -\frac{1}{K} h_i h_j^{\top}, \qquad i \neq j.$$

More precisely, the scalar covariance of the centred rewards is negative; the cross-covariance matrix is a negative scalar multiple of $h_i h_j^{\top}$. The negative dependence follows from the exact constraint

$$\sum_{i=1}^{K}(r_i - \bar{r}) = 0 :$$

if one centred reward increases, the others must collectively decrease.

(iii) The full covariance matrix is

$$\operatorname{Cov}(\hat{g}) = \frac{1}{K^2}\left[\sum_{i=1}^{K} h_i h_i^{\top} - \frac{1}{K}\left(\sum_{i=1}^{K} h_i\right)\left(\sum_{j=1}^{K} h_j\right)^{\top}\right] = \frac{1}{K^2}\sum_{i=1}^{K}(h_i - \bar{h})(h_i - \bar{h})^{\top}.$$

For the scalar variance convention used in the lectures, define

$$\mathrm{Var}_{\mathrm{tr}}(Z) := \mathrm{tr}(\mathrm{Cov}(Z)) .$$

Taking the trace gives

$$\mathrm{Var}_{\mathrm{tr}}(\hat{g}) = \frac{1}{K^2} \sum_{i=1}^{K} \|h_i - \bar{h}\|^2.$$

Also,

$$\mathrm{Cov}(X_1) = \left(1 - \frac{1}{K}\right) h_1 h_1^\top,$$

and hence

$$\mathrm{Var}_{\mathrm{tr}}(X_1) = \left(1 - \frac{1}{K}\right) \|h_1\|^2.$$

If one instead averaged K independent copies of X_1, the naive i.i.d. variance would be

$$\mathrm{Var}_{\mathrm{tr,iid}}(\hat{g}) = \frac{1}{K} \mathrm{Var}_{\mathrm{tr}}(X_1).$$

Following the lecture convention, define the effective sample size by

$$\mathrm{Var}_{\mathrm{tr}}(\hat{g}) = \frac{\mathrm{Var}_{\mathrm{tr}}(X_1)}{K_{\mathrm{eff}}}.$$

Therefore

$$K_{\mathrm{eff}} = \frac{\mathrm{Var}_{\mathrm{tr}}(X_1)}{\mathrm{Var}_{\mathrm{tr}}(\hat{g})} = \frac{K^2 \left(1 - \frac{1}{K}\right) \|h_1\|^2}{\sum_{i=1}^{K} \|h_i - \bar{h}\|^2} = \frac{K(K-1)\|h_1\|^2}{\sum_{i=1}^{K} \|h_i - \bar{h}\|^2}.$$

Equivalently, its discrepancy from the nominal sample size K is

$$\frac{K_{\mathrm{eff}}}{K} = \frac{\mathrm{Var}_{\mathrm{tr,iid}}(\hat{g})}{\mathrm{Var}_{\mathrm{tr}}(\hat{g})}.$$

If

$$\frac{1}{K} \sum_{i=1}^{K} \|h_i - \bar{h}\|^2$$

remains of constant order and $\|h_1\|^2 = O(1)$, then

$$K_{\mathrm{eff}} = \Theta(K)$$

as $K \to \infty$.

If all score vectors lie in one direction, write

$$h_i = c_i v.$$

Then

$$\mathrm{Cov}(\hat{g}) = \frac{vv^\top}{K^2} \sum_{i=1}^{K} (c_i - \bar{c})^2,$$

so the covariance has rank at most one, and

$$K_{\mathrm{eff}} = \frac{K(K-1)c_1^2}{\sum_{i=1}^{K} (c_i - \bar{c})^2}.$$

Thus only variation in the scalar coefficients c_i contributes to the variance. In the special case $h_i = h$ for every i,

$$\hat{g} = \frac{h}{K} \sum_{i=1}^{K} \hat{A}_i = 0$$

for every reward realisation. Hence $\mathrm{Cov}(\hat{g}) = 0$ and, provided $h \neq 0$, the effective sample size is formally infinite.

I.4.2 KL-regularised policy update

(i) The objective is

$$F(\pi) = \sum_a \pi(a) A_t(a) - \frac{1}{\eta} \sum_a \pi(a) \log \frac{\pi(a)}{\pi_t(a)}.$$

Introducing a Lagrange multiplier λ for $\sum_a \pi(a) = 1$, stationarity on the support of π_t gives

$$A_t(a) - \frac{1}{\eta} \left(\log \frac{\pi(a)}{\pi_t(a)} + 1 \right) + \lambda = 0.$$

Consequently,

$$\pi_{t+1}(a) = \frac{\pi_t(a) \exp(\eta A_t(a))}{\sum_b \pi_t(b) \exp(\eta A_t(b))}.$$

Actions outside the support of π_t remain assigned zero probability. The objective is strictly concave on this support, so the optimiser is unique, assuming the normalising constant is finite.

For small η, expanding $\exp(\eta A_t) = 1 + \eta A_t + O(\eta^2)$ in both the numerator and the denominator and keeping first order,

$$\pi_{t+1}(a) = \pi_t(a) \left[1 + \eta \left(A_t(a) - \mathbb{E}_{\pi_t}[A_t] \right) \right] + O(\eta^2),$$

so η controls the update size: large η places increasingly more mass on high-advantage actions.

(ii) For the true advantage of π_t,

$$J(\pi_t) = \mathbb{E}_{a \sim \pi_t}[A^{\pi_t}(a)] = 0.$$

The regularised objective evaluated at π_t is also zero. Since π_{t+1} maximises it,

$$J(\pi_{t+1}) - \frac{1}{\eta} \mathrm{KL}(\pi_{t+1} \| \pi_t) \geq 0.$$

It follows that

$$J(\pi_{t+1}) \geq \frac{1}{\eta} \mathrm{KL}(\pi_{t+1} \| \pi_t) \geq 0 = J(\pi_t).$$

Thus the update guarantees monotone improvement in the scalar performance measure defined in the question.

In neural-network GRPO the guarantee may fail because the exact advantage is replaced by a noisy finite-sample estimate and the maximisation is replaced by a finite number of stochastic-gradient steps in a restricted parametric policy class. The clipped GRPO objective is also not identical to the exact KL-regularised objective above.

(iii) For one sample, write

$$\ell(\rho, \hat{A}) = \min \left(\rho \hat{A}, \mathrm{clip}(\rho, 1 - \epsilon, 1 + \epsilon) \hat{A} \right).$$

Equivalently,

$$\ell(\rho, \hat{A}) = \begin{cases} \hat{A} \min(\rho, 1 + \epsilon), & \hat{A} \geq 0, \\ \hat{A} \max(\rho, 1 - \epsilon), & \hat{A} < 0. \end{cases}$$

Thus, for a positive advantage, the objective gives no further incentive to increase the ratio above $1 + \epsilon$. For a negative advantage, it gives no further incentive to reduce the ratio below $1 - \epsilon$.

This is not a hard constraint

$$\rho \in [1 - \epsilon, 1 + \epsilon].$$

Other samples can still move the ratio outside this interval. In contrast, a constraint

$$\mathrm{KL}\big(\pi_\theta(\cdot \mid q) \| \pi_{\mathrm{old}}(\cdot \mid q)\big) \leq \delta$$

is a constraint on the whole distribution $\pi_\theta(\cdot \mid q)$. Although this is a single distribution over the action, the divergence $\sum_a \pi_\theta(a \mid q) \log \frac{\pi_\theta(a|q)}{\pi_{\text{old}}(a|q)}$ depends on the probabilities of all actions simultaneously, including unsampled ones. PPO clipping, by contrast, acts only on the ratio at each individual sampled action and is advantage-dependent; it does not directly constrain unsampled actions, and does not guarantee that this distribution-level KL divergence is small.

I.4.3 Mixture proposal and importance weighting

We suppress the fixed prompt q from the notation. Define

$$p(a) := \pi_{\text{old}}(a \mid q), \qquad u(a) := \pi_{\text{unif}}(a \mid q), \qquad m_\alpha(a) := \pi_{\text{mix}}(a \mid q) = (1-\alpha)p(a) + \alpha u(a),$$

and

$$s_\theta(a) := \nabla_\theta \log \pi_\theta(a \mid q).$$

Assume $m_\alpha(a) > 0$ whenever $p(a) > 0$.

(i) The true policy gradient is an expectation under $p = \pi_{\text{old}}$. Importance sampling rewrites any such expectation as one under $m_\alpha = \pi_{\text{mix}}$, namely $\mathbb{E}_{a\sim p}[F] = \mathbb{E}_{a\sim m_\alpha}[w(a)\,F]$ with weight $w(a) = p(a)/m_\alpha(a)$; drawing $a_i \overset{\text{iid}}{\sim} \pi_{\text{mix}}$ therefore gives the estimator and its explicit weight

$$\widehat{g}_{\text{mix}} = \frac{1}{K}\sum_{i=1}^{K} w_i\, s_\theta(a_i)\, \widehat{A}_i, \qquad w_i = \frac{\pi_{\text{old}}(a_i \mid q)}{(1-\alpha)\,\pi_{\text{old}}(a_i \mid q) + \alpha\,\pi_{\text{unif}}(a_i \mid q)}.$$

The weight corrects the sampling distribution from m_α back to p; the correct empirical advantage $\widehat{A}_i$ is addressed in (iii).

(ii) For every fixed realised action a_i,

$$m_\alpha(a_i) \to p(a_i), \qquad w_i = \frac{p(a_i)}{m_\alpha(a_i)} \to 1.$$

Therefore

$$\widehat{g}_{\text{mix}} \to \frac{1}{K}\sum_{i=1}^{K} s_\theta(a_i)\widehat{A}_i.$$

With the standard GRPO empirical advantage

$$\widehat{A}_i = \frac{r_i - \bar{r}}{\sigma_r},$$

we obtain

$$\widehat{g}_{\text{mix}} \to \widehat{g}_{\text{GRPO}}.$$

The same statement holds if a weighted mean and weighted standard deviation are used, since $w_i \to 1$ implies that the weighted empirical statistics converge to the ordinary ones for the same realised actions.

(iii-a) Let

$$\mu_p := V^{\pi_{\text{old}}}(q), \qquad \mu_u := V^{\pi_{\text{unif}}}(q), \qquad \mu_m := V^{\pi_{\text{mix}}}(q).$$

Then, by linearity of expectation,

$$\begin{aligned} V^{\pi_{\text{mix}}}(q) &= \sum_a \big((1-\alpha)p(a) + \alpha u(a)\big) R(q,a) \\ &= (1-\alpha)V^{\pi_{\text{old}}}(q) + \alpha V^{\pi_{\text{unif}}}(q). \end{aligned}$$

Hence

$$V^{\pi_{\text{mix}}}(q) = (1-\alpha)V^{\pi_{\text{old}}}(q) + \alpha V^{\pi_{\text{unif}}}(q).$$

(iii-b) No. The importance weight w_i re-targets the sampling distribution from m_α back to p, but the baseline is still the unweighted group mean $\bar{r}$, which under mixture sampling estimates $\mu_m = V^{\pi_{\text{mix}}}(q)$, not $\mu_p = V^{\pi_{\text{old}}}(q)$. Centring with this wrong baseline and applying only the outer weight gives

$$\mathbb{E}_m[w\, s_\theta\, (R - \mu_m)] = \mathbb{E}_p[s_\theta\, (R - \mu_p)] + (\mu_p - \mu_m)\, \mathbb{E}_p[s_\theta],$$

whose first term is the target π_{old} gradient and where $\mu_p - \mu_m = \alpha(\mu_p - \mu_u)$. The residual $(\mu_p - \mu_m)\, \mathbb{E}_p[s_\theta]$ vanishes only when $\mathbb{E}_p[s_\theta] = 0$, which the score identity gives exactly at $\theta = \theta_{\text{old}}$ (where $p = \pi_\theta$) but not after any update step. So w alone is enough only at $\theta = \theta_{\text{old}}$; in general the baseline must be re-estimated under p.

Using the same weights, the old-policy baseline can be estimated by

$$\widehat{\mu}_p^{\text{IS}} = \frac{1}{K} \sum_{j=1}^{K} w_j r_j \qquad \text{or} \qquad \widehat{\mu}_p^{\text{SN}} = \frac{\sum_{j=1}^{K} w_j r_j}{\sum_{j=1}^{K} w_j}.$$

The IS form is unbiased, since the weight undoes the change of sampling law:

$$\begin{aligned} \mathbb{E}\big[\widehat{\mu}_p^{\text{IS}}\big] = \mathbb{E}_m[wR] &= \sum_a m_\alpha(a)\, \frac{p(a)}{m_\alpha(a)}\, R(q,a) \\ &= \sum_a p(a)\, R(q,a) = \mu_p, \end{aligned}$$

while $\widehat{\mu}_p^{\text{SN}}$ is consistent ($\mathbb{E}_m[w] = 1$) but slightly biased at finite K. With the analogously weighted $\widehat{\sigma}_p$, the reweighted advantage that keeps $\widehat{g}_{\text{mix}}$ unbiased for the π_{old} gradient is then

$$\widetilde{A}_i = w_i \frac{r_i - \widehat{\mu}_p}{\widehat{\sigma}_p}, \qquad \widehat{g}_{\text{mix}} = \frac{1}{K} \sum_{i=1}^{K} s_\theta(a_i)\, \widetilde{A}_i.$$

(iv) Let

$$A_p(a) := R(q,a) - \mu_p, \qquad Z(a) := s_\theta(a) A_p(a), \qquad g := \mathbb{E}_p[Z(a)].$$

Ignoring empirical baseline and variance estimation, the old-policy estimator has covariance

$$\text{Cov}(\widehat{g}_{\text{GRPO}}) = \frac{1}{K} \left(\mathbb{E}_p[ZZ^\top] - gg^\top \right).$$

The importance-sampled estimator has covariance

$$\begin{aligned} \text{Cov}(\widehat{g}_{\text{mix}}) &= \frac{1}{K} \left(\mathbb{E}_m[w^2 ZZ^\top] - gg^\top \right) \\ &= \frac{1}{K} \left(\mathbb{E}_p[wZZ^\top] - gg^\top \right). \end{aligned}$$

Therefore

$$\text{Cov}(\widehat{g}_{\text{mix}}) - \text{Cov}(\widehat{g}_{\text{GRPO}}) = \frac{1}{K} \mathbb{E}_p[(w-1)ZZ^\top].$$

There is no universal ordering: importance sampling can either increase or decrease variance.

For the scalar advantage contribution alone,

$$\text{Var}_m(wA_p) - \text{Var}_p(A_p) = \mathbb{E}_p[(w-1)A_p^2].$$

For $\alpha > 0$,

$$w(a) > 1 \iff m_\alpha(a) < p(a) \iff u(a) < p(a),$$

and

$$w(a) < 1 \iff m_\alpha(a) > p(a) \iff u(a) > p(a).$$

Therefore, if large-magnitude advantages occur mainly on actions satisfying $u(a) < p(a)$, then $w(a) > 1$ where $A_p(a)^2$ is large, so

$$\mathbb{E}_p[(w-1)A_p^2] > 0,$$

and the variance increases.

Conversely, if large-magnitude advantages occur mainly on actions satisfying $u(a) > p(a)$, then $w(a) < 1$ where $A_p(a)^2$ is large, so

$$\mathbb{E}_p[(w-1)A_p^2] < 0,$$

and the variance decreases.

(v-a) Since $0 < c < 1$ we have $\log c < 0$, so

$$w_i = c^T = e^{T \log c} \longrightarrow 0 \text{ exponentially fast as } T \to \infty.$$

A value $c < 1$ is the typical situation when $\alpha > 0$ because the responses are drawn from π_{mix}, whose uniform component inflates the probability of tokens that are unlikely under π_{old}, so the sampled tokens usually satisfy $\pi_{\text{mix}} > \pi_{\text{old}}$, i.e. $c = \pi_{\text{old}}/\pi_{\text{mix}} < 1$.

(v-b) When $w_i = c^T$ is exponentially small, almost every sampled trajectory contributes negligible gradient signal, so $\widehat{g}_{\text{mix}}$ is dominated by a handful of rare large-weight trajectories: the effective sample size collapses and the variance explodes, making the estimate unreliable for long responses.

One concrete fix is to shrink the mixture weight with length, taking $\alpha = \alpha_T = O(1/T)$ so that the per-token ratio satisfies $c_T = 1 - \kappa/T + o(1/T)$ and the full-response weight stays bounded away from zero,

$$w_i = c_T^T \longrightarrow e^{-\kappa} > 0.$$

The trade-off is weaker uniform exploration on long responses.

Part II — Adaptive planning in `cmbagent_lg`

Studied in full at commit `d7d0592` (`adaptive-planning` branch), module `cmbagent_lg/adaptive_planning` (graph, nodes, state, schemas, prompts), which composes the `planning/` and `deep_research/` modules.

II.1 Workflow diagram

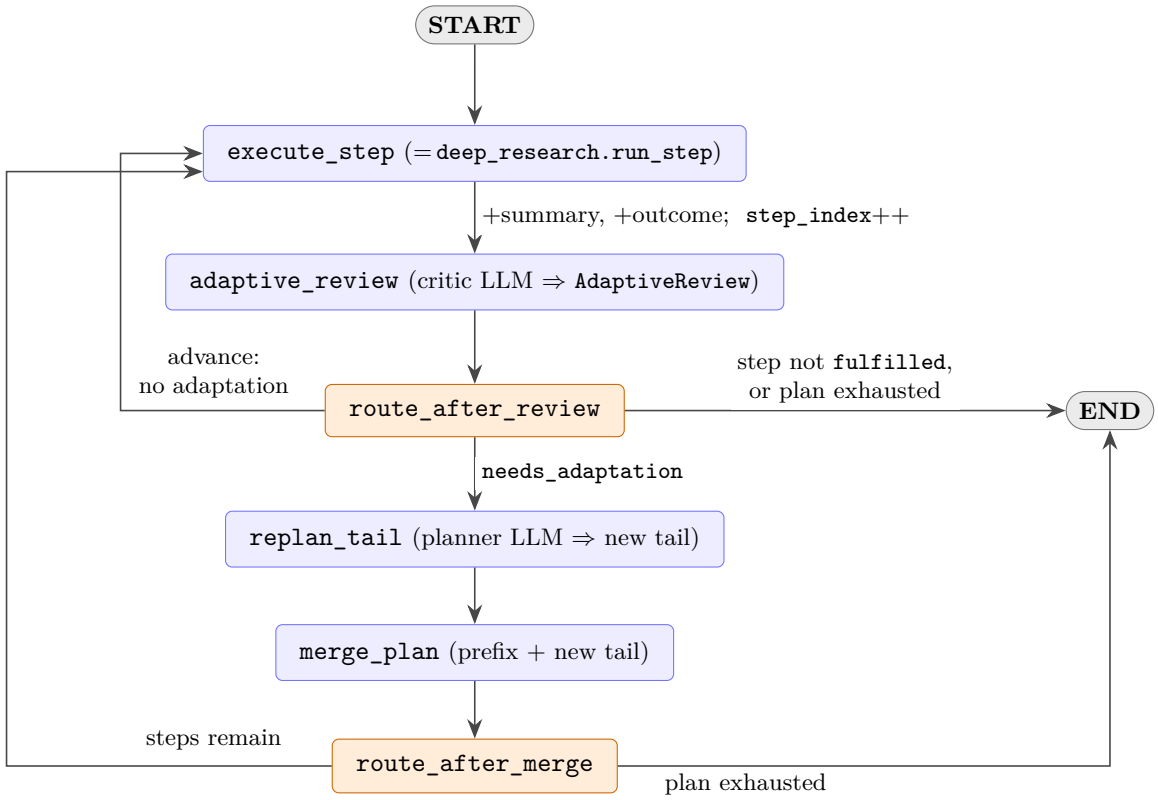


Figure 3: Adaptive-planning LangGraph graph (`adaptive_planning/graph.py`). Blue = work nodes, orange = conditional routers; the threaded `AdaptivePlanState` carries `plan`, `step_index`, `step_summaries`/`step_outcomes`, `adaptive_review`, `new_tail`, `adaptive_history`. *(Excluded from the 2-page limit.)*

Walkthrough. The initial `Plan` comes from the bounded propose–critique loop in `planning/`; this graph executes it. `execute_step` (`deep_research`'s `run_step`, reused verbatim) runs the step at `step_index`, appends a summary and a structured outcome {`fulfilled`,...}, increments `step_index`, and always passes to `adaptive_review`. The four things the brief asks for:

What triggers a revision: `adaptive_review` — a critic LLM given the overall task, the latest step's summary+outcome, and the unrun steps — returns `AdaptiveReview(needs_adaptation, recommendations)`. `route_after_review` branches to `replan_tail` *iff* the step was `fulfilled`, the plan is not exhausted, *and* `needs_adaptation` is set; otherwise it advances to the next step or halts at `END`.

What may change: only the *tail* `plan.sub_tasks[n_done:]`: `replan_tail` asks the planner LLM (given the completed summaries, the current tail, and the recommendations) for a replacement tail, and `merge_plan` sets `plan = completed_prefix + new_tail`.

What is held fixed: the completed prefix `plan.sub_tasks[:n_done]`, its stored summaries/outcomes, and `step_index` are never touched by a revision, so executed work is neither re-run nor re-ordered.

How it terminates: a router returns `END` as soon as a step is *not* `fulfilled` or the plan is exhausted (`step_index` > `len(plan)`); `execute_step` advances `step_index` by one per step while a revision never does, so the plan is consumed only by execution and the loop halts once the replanner stops extending the tail. The sole hard backstop is LangGraph's default `recursion_-`

`limit`, which *raises* rather than finishing — whether this is a genuine guarantee is taken up in II.2.

II.2 Problems

P1 — Termination is not *guaranteed*. The graph *does* have a stopping rule — `route_after_review` and `route_after_merge` return `END` the moment a step is not `fulfilled` or the plan is exhausted (`step_index > len(plan)`), so a well-behaved run does terminate. The defect is that nothing *forces* that rule to be reached: `step_index` advances only in `execute_step`, whereas `replan_tail` may return a tail *longer* than the steps it replaces and `merge_plan` routes straight back into execution. With no revision budget, no length cap and no monotone quantity that must strictly decrease, the plan can grow without bound and the "exhausted" condition recede forever. Closed-loop control is well-behaved only when the feedback law contracts toward the goal (a Lyapunov/termination argument), and a free-text "keep the tail minimal" instruction to an LLM is no such contraction. The only hard backstop, `recursion_limit`, surfaces as an *exception*, not a finished run — and the bounded-loop safeguard the initial planner relies on (`num_rounds`) was dropped here.

P2 — Adaptation is scoped to successful steps, not to recovery. The loop revises only after a *successful* step: `route_after_review` tests `fulfilled` *before* `needs_adaptation`, so a `fulfilled=False` outcome routes to `END` (the `adaptive_review` verdict, though computed, then goes unused). This is a coherent design — adaptation here is meant to make an *already-working* execution more responsive to what it has learned, not to repair a failing one. The limitation is that the closed-loop benefit (using feedback to correct course) stays confined to the success path: a failed step simply ends the run, with no retry or route-around. Where a failed step is often recoverable, extending adaptation to that case (II.3) would widen the benefit.

P3 — The replanned tail bypasses the reviewer that gates the initial plan. `planning/` accepts a plan only after a bounded `planner`↔`plan_reviewer` propose–critique loop. The adaptive replan has *no* reviewer: `merge_plan` splices the planner's `new_tail` unchecked. Revisions — generated mid-run under the pressure of a surprising result, i.e. exactly when error is likeliest — thus receive *weaker* quality control than the original plan, and can drift from `main_task` or contradict the frozen prefix with nothing to catch it.

P4 — Partial-state feedback at unconditional cost. `adaptive_review` sees only the *last* step's summary+outcome and the remaining steps — not the full trajectory or the artifacts produced — so its decision is conditioned on the last step alone (effectively Markov) and cross-step drift is invisible; closed-loop control needs full-state feedback, not a single observation. It also fires after *every* step regardless of need (and the planner on every revision), so token cost and latency scale with plan length with no gating.

II.3 Proposed improvements

P1 → bound the loop. Thread a `revision_budget` (and/or `max_steps`) through `AdaptivePlanState`, decrement it in `merge_plan`, and have the routers *finish gracefully* (run out the current tail, then `END`) once it hits zero; optionally reject any `new_tail` that would push the plan past a length cap. *Why:* reinstates a strictly-decreasing quantity, so termination no longer rests on LLM goodwill. *Trade-off:* a hard cap can truncate a genuinely expanding task — so surface "budget exhausted" as an explicit, inspectable outcome rather than a silent stop.

P2 → add a recovery path on failure. On `fulfilled=False`, route to `replan_tail` (or a dedicated `repair` node) with a per-step retry budget, halting only after K failed attempts. *Why:* extends the closed-loop benefit to the failure case — a recoverable error drives a re-plan rather than ending the run. *Trade-off:* extra execution/LLM cost and a risk of looping on an impossible step, both bounded by the retry budget; needs a recoverable-vs-fatal flag in the outcome.

P3 → validate the replanned tail. Before `merge_plan`, pass `new_tail` through the existing `planning` `plan_reviewer` for one critique round against `main_task` and the frozen prefix, regenerating if it diverges. *Why:* gives revisions the same quality gate as the initial plan, closing the asymmetry in P3. *Trade-off:* one extra reviewer call (and possibly a planner re-pass) per revision — bound it to a

single round to cap added latency.

P4 → richer, gated feedback. Give the reviewer the full step history and original goal (global state), and *gate* the call — use a cheap model (or skip it) when the outcome shows no downstream impact, escalating to the full critic only on flagged steps. *Why:* full-state feedback catches multi-step drift; gating removes the per-step tax. *Trade-off:* more context costs tokens, and gating risks missing a non-obvious adaptation — so gate conservatively.